SUMMARY: FUNCTIONS

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what functions are
- Understand how to define functions, their parameters, and arguments.
- Understand the return statement
- Understand scope and namespaces

KEY POINTS AND EXAMPLES:

Understand what functions are

Functions in Python are blocks of reusable code that perform a specific task. They allow you to organize code into manageable chunks, making it easier to understand, debug, and maintain. Functions promote code reusability and help in avoiding repetition.

Functions in Python can be compared to recipes in cooking. Just like a recipe breaks down the process of making a dish into steps, a function breaks down a task in your code into smaller, more manageable parts.

Imagine you're baking a cake. You have a recipe that tells you what ingredients you need and what steps to follow to bake the cake. Similarly, when you define a function in Python, you specify what inputs it needs (parameters) and what actions it should perform.

Once you've defined a function (or written a recipe), you can reuse it multiple times without having to rewrite the code each time. This is similar to how you can use the same recipe to bake the same cake multiple times without having to reinvent the steps.

Just as a recipe can have optional ingredients or variations, functions in Python can have default parameters or be called with different arguments to customize their behavior. And just like you can combine multiple recipes to create a new dish, you can combine multiple functions to create more complex programs.

Mainly, there are two types of functions:

1. User-defined functions – These functions are defined by the user to perform a specific task.
2. Built-in functions – These functions are pre-defined functions in Python.

User-defined functions

- User-defined functions are the functions that you create as a programmer to perform specific tasks in your code.
- These functions are defined using the def keyword.

Built-in functions

- Built-in functions are the functions that are pre-defined and come bundled with the Python language itself.
- Examples: print(), len(), max(), min(), sum(), range(), list(), str(), int(), float(), etc.
- They perform common and essential operations, such as input/output, data type conversions, mathematical operations, and more.
- Their functionality is predefined and cannot be modified.
- They are generally faster and more efficient than user-defined function

The main difference is that user-defined functions are created by you, the programmer, to solve specific problems in your code, while built-in functions are pre-existing functions provided by the Python language itself for common tasks and operations.

User-defined functions allow you to modularize and organize your code, promote code reuse, and encapsulate specific functionality. Built-in functions, on the other hand, provide a wide range of essential operations that are readily available and optimized for performance.

Understand how to define functions, their parameters, and arguments

In Python, we define a function using the def keyword followed by the function name, parentheses () for parameters, and a colon :. The function body is indented and contains the statements that the function executes when called.
Syntax:

```
def function_name(parameters):
    # Function body: Code to execute when the function is called
    # Use parameters to perform operations
    # Return a value if necessary

# Call the function
function_name(arguments)
```

- def: Keyword used to define a function.

- function_name: Name of the function, which should follow Python's naming conventions.
- parameters: Optional input data that the function can accept. Parameters are defined within parentheses () and separated by commas.
- Colon denotes the beginning of the function body.
- return: Keyword used to return a value from the function. It is optional and can appear multiple times in the function body. If omitted, the function returns None.
- arguments: Actual values passed to the function when it is called. Arguments are supplied within parentheses () and separated by commas.

```
def my_function():
  print("Hello from a function")

my_function()
```

The output will be:

```
Hello from a function
```

When you call the my_function() function, it executes the code inside the function.

```
def greet(name):
    print("Hello, " + name + "!")
```

Parameters are the placeholders for the values that a function needs to perform its task. They are specified inside the parentheses in the function definition. In the example above, name is a parameter.

```
greet("Alice")
```

When we call a function, we provide values for its parameters. These values are called arguments. In the function call, the arguments are passed within the parentheses. You can add as many arguments as you want, just separate them with a comma.

In this call, "Alice" is the argument passed to the name parameter.

```
def greet(name):
    print("Hello, " + name + "!")
greet("Alice")
```

The output will be:

```
Hello, Alice!
```

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

The output will be:

```
Emil Refsnes
Tobias Refsnes
Linus Refsnes
```

a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil", "Refsnes")
```

The output will be:

```
Emil Refsnes
```

If you try to call the function with 1 or 3 arguments, you will get an error:

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil")
```

The output will be:

```
    my_function("Emil")
TypeError: my_function() missing 1 required positional argument:
'lname'
```

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a tuple of arguments, and can access the items accordingly. This convention is often referred to as using *args."

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

The output will be:

```
The youngest child is Linus
```

Using the * before the parameter name in the function definition allows the function to accept any number of arguments. In this example, *kids receives a tuple of arguments, and within the function, you can access the individual elements of this tuple as needed. In this case, kids[2] accesses the third element of the tuple, which corresponds to "Linus".

Note: Arbitrary Arguments are often shortened to *args in Python documentations.

You can also send arguments with the key = value syntax. This way the order of the arguments does not matter. This convention is often referred to as using Keyword Arguments.

```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

The output will be:

```
The youngest child is Linus
```

The function my_function takes three parameters (child3, child2, and child1), but they are

passed in a different order compared to their definition in the function signature. The output confirms that the youngest child, as determined by the value passed to child3, is "Linus".

Note: Arbitrary Keyword Arguments are often shortened to **kwargs in Python documentations.

The following example shows how to use a default parameter value. If we call the function without argument, it uses the default value.

```
def my_function(country = "Norway"):
  print("I am from " + country)

my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

The output will be:

```
I am from Sweden
I am from India
I am from Norway
I am from Brazil
```

- When my_function("Sweden") is called, it substitutes the default value, and "Sweden" is printed.
- Similarly, when my_function("India") is called, "India" is printed.
- When my_function() is called without any arguments, the default value "Norway" is used, resulting in "Norway" being printed.
- Finally, my_function("Brazil") substitutesthe default value with "Brazil", resulting in "Brazil" being printed.

```
def function(a,b=10,c):
    return a+b-c
value = function(10,12)
print(value)
```

The output will be:

```
SyntaxError: non-default argument follows default argument
```

The error occurs because in the function definition def function(a, b=10, c):, you have a default parameter b=10 followed by a non-default parameter c.

Note: In Python, all parameters with default values must come after parameters without default values.

To correct this error, we should reorder the parameters so that parameters without default values come before parameters with default values:

```
def function(a, c, b=10):
    return a + b - c

value = function(10, 12)
print(value)
```

The output will be:

```
8
```

```
def calculate_result(x, y, z=1, w=5):
    return x + y + z + w
result = calculate_result(1, 2, w=7)
print(result)
```

The output will be:

```
11
```

- We define a function calculate_result() with four parameters: x, y, z, and w.
- Parameters c and d have default values of 1 and 5, respectively.
- When calling the function calculate_result(1, 2, w=7), we provide values for x and y, and we specify a value for w using keyword argument.
- The function adds all the provided values (x, y, z, and w) and returns the result.
- The output of the function call will be 1 + 2 + 1 + 7 = 11.
- We print the result, which is 11.

We can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

```
def my_function(food):
```

```
  for x in food:
    print(x)

fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

The output will be:

```
apple
banana
cherry
```

In this example, the function my_function accepts a single argument food, which is expected to be an iterable data type such as a list. Inside the function, it iterates over each element of the food list and prints it. When the function is called with the list fruits, each item in the list ("apple", "banana", "cherry") is printed on a separate line.

function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

```
def myfunction():
  pass
```

Understand the return statement

The return statement is used to exit a function and return a value. If no return statement is present, the function returns None by default.

```
def square(num):
    return num ** 2
result = square(5)
print(result)
```

The output will be:

```
25
```

This function, named square, takes a number as input and returns the square of that number. In this example, square(5) is called, which passes 5 as an argument to the function. Inside the function, num is squared using the ** operator, and the result is returned. The returned value, 25, is stored in the variable result, and then printed to the console.

Difference between print() and return statement

In Python, print and return statements serve different purposes, even though they may sometimes produce similar outputs.

Imagine you have a friend who knows how to bake cakes. Let's call this friend the "baking function".

print() is like your friend showing you the cake they baked.

When your friend uses print(), they are just displaying or showing you the cake.
You can see the cake, but you can't take it with you or use it for anything else.
It's like your friend saying, "Look, I baked this cake!"

return is like your friend giving you the cake.

When your friend uses return, they are actually giving you the cake.
You can take the cake with you, eat it, share it, or use it for other purposes.
It's like your friend saying, "Here, take this cake!"

So, print() is just for displaying or showing something, while return is for giving you something that you can use later.

In a Python function:

- print() is used to display or output something to the console or screen.
- return is used to give back a value that the function calculated or processed, so that value can be used elsewhere in your program.

You can use both print() and return in the same function if needed:

print() to display some intermediate results or messages.
return to give back the final result of the function.

But if you only want to display something, you use print(). If you want to get a value back from the function to use later, you use return.

```
# Using print statement
def add_with_print(a, b):
    print("Calculating the sum of", a, "and", b)
```

```
    print("The sum is:", a + b)

# Using return statement
def add_with_return(a, b):
    return a + b

# Calling the functions
add_with_print(3, 5)
result = add_with_return(3, 5)
print("The sum is:", result)
```

The output will be:

```
Calculating the sum of 3 and 5
The sum is: 8
The sum is: 8
```

In the add_with_return function, we use the return statement to pass the computed sum back to the caller, allowing it to be assigned to a variable (result in this case) and used elsewhere in the program.

```
def max_of_three(a, b, c):
    if a >= b and a >= c:
        return a
    elif b >= a and b >= c:
        return b
    else:
        return c

# Example usage
x, y, z = 10, 25, 7
maximum = max_of_three(x, y, z)
print(f"The maximum of {x}, {y}, and {z} is {maximum}")
```

The output will be:

```
The maximum of 10, 25, and 7 is 25
```

```
def celsius_to_fahrenheit(celsius):
    fahrenheit = (celsius * 9/5) + 32
```

```
    return fahrenheit

# Example usage
temp_celsius = 25.0
temp_fahrenheit = celsius_to_fahrenheit(temp_celsius)
print(f"{temp_celsius}°C is equal to {temp_fahrenheit}°F")
```

The output will be:

```
25.0°C is equal to 77.0°F
```

```
def calculate_total_cost(prices):
    total_cost = sum(prices)
    return total_cost

# Somewhere else in your program:
cart_prices = [10, 20, 30, 40]
total = calculate_total_cost(cart_prices)
print("Total cost:", total)
```

The calculate_total_cost function takes a list of prices as input, calculates the total cost by summing them up, and returns the result.

is_prime function checks whether a given integer n is a prime number.

```
def is_prime(n):
    # Check if n is less than 2, which means it's not a prime
number
    if n <= 1:
        return False
# Check for factors from 2 to the square root of n
        for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

print(is_prime(17))  # True
print(is_prime(18))  # False
```

If the input number n is less than or equal to 1, the function immediately returns False because prime numbers are defined as natural numbers greater than 1.

The function iterates over a range of numbers from 2 to n - 1 (inclusive). It skips n itself because any number is divisible by itself, so checking n % n is redundant.

Within the loop, for each number i in the range, the function checks if n is divisible by i using the modulo operator (%). If the remainder is 0, it means n is divisible by i, so n is not a prime number. In this case, the function returns False.

If the loop completes without finding any factors, it means that n has been checked against all potential factors up to n - 1, and none of them divide n evenly. In other words, no number between 2 and n - 1 is a factor of n. Therefore, to signify that n is a prime number, the function returns True after the loop completes.

To determine if a given number n is a palindrome

```
def is_palindrome(n):
    # Convert the number to a string
    str_n = str(n)
    # Check if the string is equal to its reverse
    return str_n == str_n[::-1]

# Example usage:
n = 12321
print(is_palindrome(n))  # Output: True

n = 12345
print(is_palindrome(n))  # Output: False
```

- We use str(n) to convert the integer n to its string representation.
- We compare the string str_n with its reverse str_n[::-1]. In Python, str_n[::-1] creates a reversed copy of the string str_n.
- If the string and its reverse are equal, the function returns True, indicating that the number is a palindrome. Otherwise, it returns False.

To determine if a given number n is a armstrong number
(An Armstrong number (also known as a narcissistic number or pluperfect number) for a given number of digits is a number that is equal to the sum of its own digits each raised to the power of the number of digits. For example, 153 is an Armstrong number because 1^3+5^3+3^3=153)

```
def is_armstrong(n):
```

```
    # Convert the number to a string to easily iterate over digits
    str_n = str(n)
    # Calculate the number of digits
    num_digits = len(str_n)
    # Calculate the sum of the digits each raised to the power of
num_digits
    sum_of_powers = sum(int(digit) ** num_digits for digit in
str_n)
    # Check if the sum of powers is equal to the original number
    return sum_of_powers == n

# Example usage:
n = 370
print(is_armstrong(n))  # Output: True

n = 123
print(is_armstrong(n))  # Output: False
```

- We use str(n) to convert the integer n to its string representation so that we can easily iterate over its digits.
- len(str_n) gives us the number of digits in n.
- We use a generator expression within the sum() function to iterate over each digit, convert it back to an integer, raise it to the power of num_digits, and sum these values.
- Finally, we compare the sum of the powers with the original number n. If they are equal, n is an Armstrong number, and the function returns True. Otherwise, it returns False.
- For n = 370, the calculation will be 3^3+7^3+0^3=370, so the function returns True.
- For n = 123, the calculation will be 13+23+33=36 , so the function returns False.

To calculate the greatest common divisor (GCD) of two numbers x and y
(GCD refers to the largest positive integer that divides two or more numbers without leaving a remainder.)

```
def gcd(x, y):
    while y != 0:
        x, y = y, x % y
    return x

# Example usage:
x = 48
y = 18
```

```
result = gcd(x, y)
print("GCD of", x, "and", y, "is:", result)
```

The output will be:

```
GCD of 48 and 18 is: 6
```

1. We define the gcd function that takes two integers x and y as input.
2. The loop continues until y becomes 0
3. Inside the loop, we update x and y using the statement x,y=y,x%y. This means:
    - x is updated to the current value of y.
    - y is updated to the remainder of x divided by y.
4. Once the loop exits (when y becomes 0), x contains the GCD of the original x and y, which we then return.

    - For x=48 and y=18:
        - The first iteration:
        - x=18, y=48%18=12
        - The second iteration:
        - x=12, y=18%12= 6
          The third iteration:
        - x= 6, y=12%6= 0
        - Loop exits and the function returns x= 6

Note: Using the return statement outside of a function definition will raise a SyntaxError, because the return statement is only valid within a function body.

```
# This will raise a SyntaxError
return "Hello, World!"
```

In Python, functions can return multiple values by separating them with commas., and it will implicitly create a tuple.

```
def get_values():
    return 1, 2, 3

values = get_values()
print(values)  # Output: (1, 2, 3)
```

To calculate the sum and product of two numbers and return both values

```
def sum_and_product(a, b):
    sum_result = a + b
    product_result = a * b
    return sum_result, product_result

num1 = 6
num2 = 10
sum_result, product_result = sum_and_product(num1, num2)
print(sum_result)
print(product_result)
```

The output will be:

```
16
60
```

If you try to return a variable that is not defined within the function's scope, you'll get a NameError.

```
def my_func():
    return x  # NameError: name 'x' is not defined

my_func()
```

Passing a Function as an Argument to Another Function

```
# Define a higher-order function that applies a function to each
element in a list
def transform_list(func, values):
    return [func(value) for value in values]

# Define a function that doubles a number
def double(x):
    return x * 2

# Define a function that adds one to a number
def add_one(x):
    return x + 1
```

```
# Example usage
numbers = [1, 2, 3, 4, 5]

# Apply the 'double' function to each element in the list
doubled_numbers = transform_list(double, numbers)
print("Doubled numbers:", doubled_numbers)

# Apply the 'add_one' function to each element in the list
incremented_numbers = transform_list(add_one, numbers)
print("Incremented numbers:", incremented_numbers)
```

The output will be:

```
Doubled numbers: [2, 4, 6, 8, 10]
Incremented numbers: [2, 3, 4, 5, 6]
```

In this example:

- transform_list is a higher-order function that takes another function (func) and a list of values (values) as arguments. It applies the function to each element in the list.
- double is a function that doubles its argument.
- add_one is a function that adds one to its argument.
- We pass the double and add_one functions along with a list of numbers to transform_list.

Understand scope and namespaces

Scope: Scope refers to the region of a program where a variable is visible and can be accessed. Python has two main scopes: global scope and local scope.

1. **Global Scope:** Variables defined outside any function are considered to be in the global scope. These variables can be accessed from anywhere in the program, including within functions.

```
x = 10  # Global variable

def my_function():
    print(x)  # Accessing the global variable 'x'
my_function()  # Output: 10
```

2. **Local Scope:** Variables defined within a function are considered to be in the local

scope. These variables are only accessible within the function where they are defined.

```
def my_function():
    y = 20  # Local variable
    print(y)  # Accessing the local variable 'y'

my_function()  # Output: 20
print(y)  # NameError: name 'y' is not defined
```

When the function my_function is called, the local variable y is accessible within its scope, and the value 20 is printed successfully.

However, when attempting to print y outside the function scope, as in the line print(y), a NameError occurs. This error arises because y is a local variable and is not defined in the global scope where the print(y) statement is executed. Outside the function, the name y is not recognized, leading to the NameError.

```
x = "awesome"
def myfunc():
  x = "fantastic"
  print("Python is " + x)

myfunc()

print("Python is " + x)
```

The output will be:

```
Python is fantastic
Python is awesome
```

```
a = 14
def f():
    a = 12
    return a
a = f()
print(a)
```

The output will be:

```
12
```

- Here, a global variable a is created with a value of 14.
- def f(): This line defines a function named f.
-
- Inside the function f(), a local variable a is created with a value of 12. This local variable a exists only within the scope of the function f(), and it does not affect the global variable a.
- a = f(): This line calls the function f() and assigns its return value (12) to the global variable a. This operation replaces the previous value of a (14) with the value returned by the function (12).
- Finally, the value of the global variable a (which is now 12, as assigned by the function call a = f()) is printed.

If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function. The global variable with the same name will remain as it was, global and with the original value.

To create a global variable inside a function, we can use the global keyword.

```
def myfunc():
  global x
  x = "fantastic"

myfunc()
print("Python is " + x)
```

The output will be:

```
Python is fantastic
```

Also, use the global keyword if you want to change a global variable inside a function.

```
x = "awesome"

def myfunc():
  global x
  x = "fantastic"

myfunc()
```

```
print("Python is " + x)
```

The output will be:

```
Python is fantastic
```

Namespaces: Think of a namespace as a container or a box where Python stores names (variables, functions, etc.) and their corresponding objects. Each namespace is like a separate box, and Python keeps track of which names are stored in which box.

Imagine you have a big storage room with different boxes labeled "Built-in," "Global," and "Local." These boxes represent the different namespaces in Python.

1. **Built-in Namespace:** This box contains all the names (functions, constants, etc.) that are built into the Python language itself. For example, print, len, sum, max, etc., are all stored in this box.

2. **Global Namespace:** This box is where Python stores names that you define at the top level of your script, outside of any functions. For example, if you define a variable x = 10 at the top level, it goes into the "Global" box.

3. **Local Namespace:** Every time you define a function, Python creates a new "Local" box specifically for that function. Any names (variables, nested functions, etc.) defined inside the function are stored in this "Local" box.

When Python needs to find the value associated with a name (e.g., x), it follows a specific order of searching through these boxes, known as the **LEGB rule** (Local, Enclosing, Global, Built-in):

1. Python first looks in the "Local" box of the current function.
2. If not found, it looks in the "Local" box of any enclosing functions (if the current function is nested inside another function).
3. If still not found, it looks in the "Global" box.
4. Finally, if the name is still not found, Python looks in the "Built-in" box.

If the name is not found in any of these boxes, Python raises a NameError.

For example, if you have a variable x defined globally and another variable x defined inside a function, when you try to access x inside the function, Python will look in the "Local" box of that function first. If it finds x there, it will use that value. If not, it will look in the "Global" box.

It ensures that names defined in different parts of your program don't accidentally overwrite

each other and that names are accessible only within their intended scope.

```
# Global namespace
x = 10

def outer_function():
    # Local namespace of outer_function
    y = 20

    def inner_function():
        # Local namespace of inner_function
        z = 30

        # 1. Python first looks in the local namespace of inner_function
        print(z)  # Output: 30

        # 2. If not found, it looks in the local namespace of the
enclosing outer_function
        print(y)  # Output: 20

        # 3. If still not found, it looks in the global namespace
        print(x)  # Output: 10

        # 4. Finally, if the name is still not found, Python looks in
the built-in namespace
        print(len([1, 2, 3]))  # Output: 3 (len() is a built-in
function)

        # Trying to access an undefined name will raise a NameError
        # print(undefined_name)  # NameError: name 'undefined_name' is
not defined

    inner_function()

outer_function()
```

1. Local Namespace of inner_function():
   When Python encounters z inside inner_function(), it looks in the local namespace of inner_function() and finds the value z = 30. So, it prints 30.

2. Local Namespace of outer_function():
   When Python encounters y inside inner_function(), it doesn't find y in the local namespace of inner_function(). So, it looks in the local namespace of the enclosing function outer_function() and finds the value y = 20. Thus, it prints 20.

3. Global Namespace:
   When Python encounters x inside inner_function(), it doesn't find x in the local namespaces of inner_function() or outer_function(). So, it looks in the global namespace and finds the value x = 10. Thus, it prints 10.

4. Built-in Namespace:
   When Python encounters len([1, 2, 3]) inside inner_function(), it doesn't find len in any of the local or global namespaces. So, it looks in the built-in namespace and finds the len() function, which it uses to calculate the length of the list [1, 2, 3]. Thus, it prints 3.

If you try to access an undefined name, like undefined_name in the commented line, Python will raise a NameError because it cannot find that name in any of the namespaces.

```
# Global namespace
x = 10

def outer_function():
    # Local namespace of outer_function
    x = 20  # Shadows the global 'x'
    y = 30

    def inner_function():
        # Local namespace of inner_function
        x = 40  # Shadows the outer_function's 'x'
        z = 50

        # Accessing names inside inner_function() follows the LEGB
rule
        print("Inside inner_function:")
        print(x)  # Output: 40 (Local to inner_function)
        print(y)  # Output: 30 (From outer_function's namespace)
        print(z)  # Output: 50 (Local to inner_function)

    # Accessing names inside outer_function()
    print("Inside outer_function:")
    print(x)  # Output: 20 (Local to outer_function)
    print(y)  # Output: 30 (Local to outer_function)
    inner_function()
```

```
outer_function()
```

The output will be:

```
Inside outer_function:
20
30
Inside inner_function:
40
30
50
```

1. Inside inner_function():

- When Python encounters x inside inner_function(), it finds the local variable x = 40 in the local namespace of inner_function() and uses that value.
- For y, Python doesn't find it in the local namespace of inner_function(), so it looks in the enclosing namespace, which is the local namespace of outer_function(), and finds y = 30.
- Similarly, for z, Python finds it in the local namespace of inner_function() and uses the value z = 50.

2. Inside outer_function():

    - When Python encounters x inside outer_function() but outside inner_function(), it finds the local variable x = 20 in the local namespace of outer_function() and uses that value.
    - For y, Python finds it in the local namespace of outer_function() and uses the value y = 30.

The reason why "Inside outer_function:" is printed before "Inside inner_function:" is because of the order in which the print statements are encountered during the execution of the code.

1. outer_function() is called.
2. Inside outer_function(), there is a print statement: print("Inside outer_function:"). This statement is executed immediately, printing "Inside outer_function:" to the console.
3. After that, the inner_function() is called from within outer_function().
4. Inside inner_function(), there is another print statement: print("Inside inner_function:"). This statement is executed after outer_function()'s print statement, printing "Inside inner_function:" to the console.

Note: When there are nested functions, the outer function is executed first before the inner function. This means that any code within the outer function, including the definition and execution of the inner function, will be processed before the inner function is executed.

```
def outer_function(x, y):
    def add(a, b):
        return a + b

    def multiply(a, b):
        return a * b

    def calculate(op, a, b):
        if op == "add":
            return add(a, b)
        elif op == "multiply":
            return multiply(a, b)
        else:
            return "Invalid operation"

    return calculate("add", x, y), calculate("multiply", x, y)

result = outer_function(3, 4)
print(result)  # Output: (7, 12)
```

1. outer_function takes two arguments, x and y.

2. Inside outer_function, there are three nested functions:
- add(a, b): Adds two numbers a and b.
- multiply(a, b): Multiplies two numbers a and b.
- calculate(op, a, b): Takes an operation op (either "add" or "multiply") and two numbers a and b. It then calls either add or multiply based on the operation specified.

3. outer_function calls calculate twice:
    - The first call (calculate("add", x, y)) calculates the sum of x and y using the add function.
    - The second call (calculate("multiply", x, y)) calculates the product of x and y using the multiply function.
4. Finally, outer_function returns a tuple containing the results of both calculations.

So, when you call outer_function(3, 4), it calculates the sum and product of 3 and 4, resulting in the tuple (7, 12). This output represents the sum and product of the input numbers.

```
def greet():
    print("Entering greet")
```

```
    say_hello()
    print("Exiting greet")

def say_hello():
    print("Entering say_hello")
    introduce()
    print("Exiting say_hello")

def introduce():
    print("Entering introduce")
    print("Hello, I'm a friendly AI!")
    print("Exiting introduce")

greet()
```

The output will be:

```
Entering greet
Entering say_hello
Entering introduce
Hello, I'm a friendly AI!
Exiting introduce
Exiting say_hello
Exiting greet
```

- The greet function is called first.
- Inside greet, "Entering greet" is printed.
- say_hello is called from within greet.
- Inside say_hello, "Entering say_hello" is printed.
- introduce is called from within say_hello.
- Inside introduce, "Entering introduce" is printed, followed by the message "Hello, I'm a friendly AI!".
- "Exiting introduce" is printed as introduce completes its execution.
- Control returns to say_hello, and "Exiting say_hello" is printed.
- Control returns to greet, and "Exiting greet" is printed.